

The following steps need to be completed in order to successfully deploy and run the Restaurant Manager Project

Step 1: Download the project zip file from the following Github repository

- <https://github.com/ccarolinestone/RestaurantManager.git>

Step 2: Ensure the following versions of these software and tools are installed on your computer

- Install **Visual Studio Code version 1.111** for your code editor via the Visual Studio Code website
- Download **WSL2 2.4.12.0+** and **Ubuntu 22.04.5+** to use the Ubuntu terminal in Visual Studio Code
- Download **JDK version 17+** via Visual Studio Ubuntu terminal and install **Extension Pack for Java** in Visual Studio Code
- Download **Maven version 3.6.3+** via Visual Studio Code Ubuntu terminal
- Download **MySQL 8.0.45+** via Visual Studio Code Ubuntu terminal
- Download **MySQL Workbench 8.0.46+** on your computer via the MySQL website

The following software and tools are also needed for this project but are Maven dependencies listed in the pom.xml file of the downloaded project zip file. These software and tools will be automatically downloaded onto your computer when you are instructed to run the `mvn spring-boot:run` command:

- **Spring Boot version 3.5.11**, which is the framework of the application and includes **Tomcat**, which is the server of the application
- **Spring Boot Starter Data JPA**, which is the framework used for the data access layer
- **Spring Boot Starter Web**, which is used to create the REST APIs of the project
- **Spring Boot Starter Validation**, which is used to check if data meets the standards set in the project
- **Spring Boot Starter Test**, which is used for testing the application
- **Spring Boot Maven Plugin**, which allows Spring Boot to be accessed via Maven
- **MySQL Connect J**, which connects the Java project in Visual Studio Code with the MySQL data

Step 3: Extract the Restaurant Manager project from the zip file and open the root folder in Visual Studio Code. The extracted folder will be called “RestaurantManager-main”, double click that folder to be brought to the inner root folder called “RestaurantManager-main”. The entire contents of the project should now be visible in Visual Studio Code.

- The project is structured in the following way:
 - The top layer is the **Web Client**
 - This is what users of the Restaurant Manager will interact with
 - This layer was created with an HTML file and retrieves its data from the REST API endpoints
 - The next layer is the **Rest Controllers**
 - The REST API endpoints are stored within this layer
 - This layer can be found within the project's controller folder
 - Every table receives its own controller file
 - During a user interaction, a specific controller will determine which operation is taking place and which service file to call.
 - The next layer is the **Service Layer**
 - There is a service file for each object, which can be found in the service folder
 - Specific CRUD operations are defined in each file, which are associated with the CRUD functions in the Business Manager.
 - The specific operation taking place with the specific object helps locate a function in the service file to call. The designated function will then call the corresponding function in the Business Manager
 - The next layer is the **Business Manager**
 - This file can be found in the business folder and is titled BusinessManager.java
 - This file has CRUD operations for each object
 - These functions contain rules and checks to ensure the data being passed through is correct
 - Each of these functions calls the corresponding repository file
 - The next layer is the **JPA Repositories**
 - Each object has a repository file associated with it
 - The CRUD operation that has been determined in previous layers will be associated with a SQL query that will help find the data in the database
 - The last layer is the **restaurant_manager MySQL database**
 - The schema and data files can be found in src/main/resources folder
 - There are 7 tables with specific relationships amongst them which hold the data to make up this project

- The specific data needed for the original user interaction will be collected here and passed back to the previous layers

Step 4: Set up the schema using MySQL

- Enter the MySQL server via Visual Studio Code Ubuntu terminal using the following command: `sudo service mysql start`
 - Then, enter the password associated with the Ubuntu account to start MySQL
- Open MySQL Workbench and create a new connection that will be used for the project
 - Use the following information to create the new connection:
 - Connection Method: Standard (TCP/IP)
 - Hostname: 127.0.0.1
 - Port: 3306
 - Username: root
 - You will also create a username and password for this connection.
- Head over to Visual Studio Code to the opened “RestaurantManager-main” folder.
 - Find the file titled “application.properties” which is located at the following path: RestaurantManager-main/src/main/resources/application.properties.
 - In this file, please enter your username and password that were created for the connection.
 - Replace the username placeholder with your username at this line in the file: `spring.datasource.username=username`
 - Replace the password placeholder with your password at this line in the file: `spring.datasource.password=password`
 - Please save the “application.properties” file after making these updates
- Enter into the connection in MySQL Workbench after creating it to begin the process of setting up the database for the project
 - On the left-hand side of the page, there is an area labeled “Schemas”. Underneath the “Schemas” label, in the large blank area, right click with your mouse to reveal a menu with a tab called “Create Schema”. Select the “Create Schema” button.
 - A new tab will appear where you can name the new schema. For the name of the schema please type “restaurant_manager” (without the quotations). Then, select the “Apply” button.
 - A pop-up titled “Review the SQL Script to be Applied on the Database” will show and you will select the “Apply” button. Then, select the “Finish” button.
 - The restaurant_manager schema has been created and can now be seen on the left-hand side of the page under “Schemas”.
- To create the tables for the schema, find where it says “restaurant_manager” on the left-hand side of the MySQL Workbench page and double click it until the text becomes bold.
 - Head back to Visual Studio Code and locate the following file: RestaurantManager-main/src/main/resources/schema.sql. This file contains the

SQL code to create each of the 7 tables and their fields. Please copy the contents of the schema.sql file.

- Head back over to the MySQL Workbench, create a new SQL tab, and paste the schema.sql file contents there. Please run this file in MySQL Workbench. You should see that 7 tables were created under the Tables label on the left-hand side of the page. You may need to refresh the page to be able to see the tables.
- Head back to Visual Studio Code and locate the following file: RestaurantManager-main/src/main/resources/data.sql. This file contains the initial data that needs to be inserted into each table. Please copy the contents of the data.sql file.
 - Head back over to the MySQL Workbench, create a new SQL tab, and paste the data.sql file contents there. Please run this file in MySQL Workbench. There should be 50 rows of data inserted into every table. To check that the data was inserted into each table, please run the following commands in MySQL Workbench:
 - SELECT * FROM location;
 - SELECT * FROM restaurant;
 - SELECT * FROM dish;
 - SELECT * FROM experience;
 - SELECT * FROM want_to_try;
 - SELECT * FROM have_tried;
 - SELECT * FROM rating;

Step 5: Run the Service Console Client to test the service layer and access all CRUD operations through the REST API.

- In Visual Studio Code within the RestaurantManager-main/src/main/java/com/example/restaurant/app/ServiceConsoleClient.java file, there is a test to focus on the service layer and how all of the layers interact with each other. This test will show how the CRUD operations perform while they are being accessed through the layers of the project.
- To run this test, access the “RestaurantManager-main” root folder of the Ubuntu terminal of Visual Studio Code:
 - In one terminal, run the following commands:
 - `mvn -DskipTests clean package`
 - `mvn spring-boot:run`
 - When these commands are run, the Tomcat server will start, which you will see appear in the terminal.
 - To identify if the Tomcat server is running properly, look for a line in the terminal that states that the Tomcat server started on port 8080.
 - Then, in another Ubuntu terminal at the root folder, run the following command:

```
mvn -q exec:java
-Dexec.mainClass="com.example.restaurant.app.ServiceConsoleClient"
-Dexec.classpathScope=runtime
```

- This test will walk through 4 steps to highlight the 4 CRUD operations. Objects are created, objects are retrieved, objects are updated, and objects are deleted. The GET function is called frequently throughout this test to show the current state of the data. The test is printed out in the same terminal as this command, so the results are on display.
 - To determine if this test was successful, the test should display a starting message at the top of the output and a complete message at the bottom of the output. Additionally, if you follow along with each CRUD operation being implemented, you will see how the data changes.

To view each of the REST API endpoints for each of the 7 objects and to view the services of the service layer:

- Ensure the Tomcat server is still running in the terminal where the following command was run: `mvn spring-boot:run`
- In a web browser, type in the following URLs one by one to view each service:
 - `http://localhost:8080/api/locations`
 - `http://localhost:8080/api/restaurants`
 - `http://localhost:8080/api/wanttotry`
 - `http://localhost:8080/api/havetried`
 - `http://localhost:8080/api/ratings`
 - `http://localhost:8080/api/experiences`
 - `http://localhost:8080/api/dishes`
- At each URL, you will be able to see all of the JSON data representing the data held within each MySQL table. This data is not coming directly from MySQL, but the other layers in this project help to bring that information forward to display it.

Step 6: Use Spring Boot and Maven to run the application and ensure front-end works

- In Visual Studio Code within the “RestaurantManager-main” root folder of the Ubuntu terminal, please run the following commands:
 - `mvn -DskipTests clean package`
 - `mvn spring-boot:run`
- When these commands are run, the Tomcat server will start, which you will see appear in the terminal.
 - To identify if the Tomcat server is running properly, look for a line in the terminal that states that the Tomcat server started on port 8080.
- The Restaurant Manager project is hosted locally on your computer through Spring Boot and Tomcat. The web client was created using HTML and can be found at `RestaurantManager-main/src/main/resources/static/index.html` in Visual Studio Code. The web client is able to access the service layer and REST API through using the Fetch

API in JavaScript. To access the locally hosted front-end, you can type the following into your web browser: <http://localhost:8080>. Once arriving at the website: select the “Fetch All” button to load the data.

- Through the front-end, you can interact with the MySQL database through all CRUD Operations. There are 7 tabs at the top of the website where you can access the data in each of the 7 tables.
 - In every tab, you are able to edit and delete any pre-existing entry in the table through the “Edit” and “Delete” buttons on the sides of each entry. Additionally, you can add a new entry to every table through the Add New box at the top of each tab and selecting the “Create” button. After creating a new entry, if you scroll to the bottom of the page, you will see the new entry. Each tab also offers ways to search and filter the data to retrieve specific entries from the table. You may need to refresh the page by pressing the “Fetch All” button to see any new additions or updates to the data.
 - For the Location Tab:
 - There are numerous cities and states already entered into the database. The location represents where a restaurant is located. Additionally, you can filter by the ID associated with each city and state. To verify any changes made to the Location table, you can go back into MySQL Workbench and run the following commands:
 - `USE restaurant_manager;`
 - `SELECT * FROM location;`
 - For the Restaurant Tab:
 - Each restaurant has a unique ID, name, type, cuisine style, and location in the frontend. In MySQL, each restaurant has an id, restaurant name, location ID, type, and cuisine. You can also search by restaurant ID and location of the restaurant in the frontend. To verify any changes made to the Restaurant table, you can go back into MySQL Workbench and run the following commands:
 - `USE restaurant_manager;`
 - `SELECT * FROM restaurant;`
 - For the Want To Try Object:
 - Each Want To Try object represents a restaurant that the user wants to go to. The Want To Try object consists of an ID and a restaurant in the frontend. In MySQL, each want_to_try entry has an id and a restaurant ID. To verify any changes made to the Want To Try table, you can go back into MySQL Workbench and run the following commands:
 - `USE restaurant_manager;`
 - `SELECT * FROM want_to_try;`
 - For the Have Tried Object:

- Each Have Tried object represents a restaurant the user has eaten at. The Have Tried object consists of an ID and a restaurant in the front end. In MySQL, each have_tried entry has an ID and a restaurant ID. To verify any changes made to the Have Tried table, you can go back into MySQL Workbench and run the following commands:
 - USE restaurant_manager;
 - SELECT * FROM have_tried;
- For the Rating Object:
 - Each Rating object is used to rate a restaurant a user has eaten at. The Rating object in the frontend consists of an ID, the Have Tried restaurant, the food rating, the vibe rating, and a description of the experience. In MySQL, each rating entry has an ID, the ID of the Have Tried restaurant, a food rating, a vibe rating, and a rating description. To verify any changes made to the Rating table, you can go back into MySQL Workbench and run the following commands:
 - USE restaurant_manager;
 - SELECT * FROM rating;
- For the Experience Object:
 - Each Experience object is used to keep track of when a user went to a specific restaurant. The Experience object in the frontend consists of an ID, the Have Tried restaurant, the date of attendance, and a description of the time there. In MySQL, each experience entry has an ID, an ID of the Have Tried restaurant, the date, and the description. To verify any changes made to the Experience table, you can go back into MySQL Workbench and run the following commands:
 - USE restaurant_manager;
 - SELECT * FROM experience;
- For the Dish Object:
 - Each Dish object represents what was eaten at each restaurant. The Dish object in the frontend consists of an ID, the Have Tried restaurant, the dish name, and a description of the dish. In MySQL, each dish entry has an ID, the ID of the Have Tried restaurant, the dish name and a description of the food. To verify any changes made to the Dish table, you can go back into MySQL Workbench and run the following commands:
 - USE restaurant_manager;
 - SELECT * FROM dish;

